

Database Document

The Cloud Firestore data model for Raj Raani Collections — every collection, its fields, sample documents, required indexes, and the data-integrity habits the app is built around.

Brand	Raj Raani Collections
Database	Cloud Firestore, region asia-south1 (Mumbai)
Security rules	See the companion Security Rules & Compliance Document

01

Document database, not relational

Firestore stores collections of documents, not tables of rows. There are no joins — a screen that needs data from two places either nests it as a subcollection, denormalizes a copy of the field it needs, or makes two reads. The model below makes that trade-off deliberately in a couple of places, each one called out.

02

Collection map

```
users/{uid}                fullName, phone, email, role, createdAt
|-- addresses/{addressId}  fullName, phone, line1, line2, city, state, pincode,
  isDefault
|-- cart/{itemId}          productId, variantId, quantity
|-- wishlist/{productId}   addedAt

categories/{categoryId}     name, slug, description, imageUrl, sortOrder, isActive

products/{productId}        categoryId, name, slug, description, price,
  compareAtPrice,
                                fabric, workType, setIncludes, weightGrams,
                                isCustomizable, leadTimeDays,
                                isFeatured, isBestSeller, status, images: [{url,
                                alt}]

productVariants/{variantId} productId, productName, categoryId, size, color, sku,
  stock, lowStockThreshold, priceOverride, updatedAt

orders/{orderId}            orderNumber, userId, status, subtotal, deliveryCharge,
  total,
                                paymentMethod, paymentStatus,
                                razorpayOrderId, razorpayPaymentId,
                                shiprocketOrderId, awbNumber, courierName,
                                shippingAddress (map), items: [{...}], placedAt

stockLogs/{logId}           variantId, productId, previousStock, newStock,
  reason, changedBy, changedAt

assistantLogs/{logId}       sessionId, userId, messages: [...], createdAt
```

03

Why productVariants is its own flat collection

Sizes and colors could have been stored as an array field inside each product document — but Firestore cannot run a range query (like "stock is at or below 3") inside an array of maps across many documents. Pulling every variant into its own top-level collection, with the parent product's name and category copied onto each row, makes the owner's inventory table a single, fast, sortable, filterable query — where `stock <= lowStockThreshold` — instead of an in-app scan of every product. The cost is a small one: if a product's name changes, the denormalized `productName` on its variants needs updating alongside it, which the admin

product editor does automatically in the same write.

04

Field reference

products/{productId}

FIELD	TYPE	NOTES
categoryId	reference	Which category this belongs to
name, slug	string	Slug drives the /product/[slug] URL — unique, lowercase, hyphenated
price, compareAtPrice	number	compareAtPrice shows a strikethrough when higher than price
fabric, workType, setIncludes	string	Silk/georgette/net · zari/zardozi/sequin · what's included in the set
weightGrams	number	What Shiprocket charges freight on
isCustomizable, leadTimeDays	boolean, number	A made-to-order piece skips the delivery-estimate badge and shows a lead time instead
isFeatured, isBestSeller	boolean	Powers Home sections and the /shop?filter= views
status	string	'draft' or 'published' — drafts never appear on the storefront
images	array of maps	{url, alt} — url points at Cloud Storage

productVariants/{variantId}

FIELD	TYPE	NOTES
productId, productName, categoryId	reference / string	Denormalized so the inventory table needs no join
size, color, sku	string	sku is the human-facing code used in Excel import/export
stock	number	The number the whole inventory system reads and writes
lowStockThreshold	number	Default 3 — anything at or below shows an amber badge
priceOverride	number, nullable	Rare — a specific size/color priced differently from the product default

orders/{orderId}

FIELD	TYPE	NOTES
orderNumber	string	Human-facing — what the customer quotes on WhatsApp or /track-order

FIELD	TYPE	NOTES
userId	reference, nullable	null for a guest order
status	string	New / Confirmed / Processing / Shipped / Delivered / Cancelled
paymentMethod, paymentStatus	string	'razorpay' or 'cod'; paymentStatus tracks pending/paid/refunded
razorpayOrderId, razorpayPaymentId	string	Set once payment is created / captured
shiprocketOrderId, awbNumber, courierName	string	Set once the order is pushed to Shiprocket
shippingAddress	map	A full copy of the address at the time of order — never a reference to a mutable address doc
items	array of maps	name and price are snapshotted at purchase time — see section 06

05

Sample documents

Realistic documents, not placeholders — this is what a working catalog and a placed order actually look like.

products/lh_royal_rose

```
{
  "categoryId": "cat_bridal",
  "name": "Royal Rose Bridal Lehenga",
  "slug": "royal-rose-bridal-lehenga",
  "price": 42999,
  "compareAtPrice": 48999,
  "fabric": "Silk with zari embroidery",
  "workType": "Zari, zardozi",
  "setIncludes": "Lehenga, unstitched blouse, dupatta",
  "weightGrams": 2600,
  "isCustomizable": true,
  "leadTimeDays": 21,
  "isFeatured": true,
  "isBestSeller": false,
  "status": "published",
  "images": [
    { "url": "https://.../royal-rose-1.jpg", "alt": "Royal Rose, front" }
  ]
}
```

productVariants/var_lh_royal_rose_m_maroon

```
{
  "productId": "lh_royal_rose",
  "productName": "Royal Rose Bridal Lehenga",
  "categoryId": "cat_bridal",
  "size": "M",
  "color": "Maroon",
  "sku": "RRC-BR-001-M-MRN",
  "stock": 3,
  "lowStockThreshold": 3,
  "priceOverride": null,
  "updatedAt": "2026-09-10T11:20:00Z"
}
```

orders/ord_20260910_0042 — orderNumber "RRC-1042"

```
{
  "orderNumber": "RRC-1042",
  "userId": null,
  "status": "Confirmed",
  "subtotal": 42999,
  "deliveryCharge": 0,
  "total": 42999,
  "paymentMethod": "razorpay",
  "paymentStatus": "paid",
  "razorpayOrderId": "order_XXXXXXXX",
  "razorpayPaymentId": "pay_XXXXXXXX",
  "shippingAddress": {
    "fullName": "Priya Sharma", "phone": "+91XXXXXXXXXX",
    "line1": "12 Sector 18", "city": "Noida", "state": "Uttar Pradesh", "pincode":
      "201301"
  },
  "items": [
    { "productId": "lh_royal_rose", "variantId": "var_lh_royal_rose_m_maroon",
      "nameSnapshot": "Royal Rose Bridal Lehenga", "priceSnapshot": 42999, "quantity":
        1 }
  ],
  "placedAt": "2026-09-10T11:22:04Z"
}
```

06

Two habits that keep the data honest

1. **Price and name are snapshotted** onto order.items at the moment of purchase. If a product's price changes next month, every past invoice still reads correctly — it is not silently rewritten.
2. **order.status is the single source of truth** for the whole fulfillment workflow: New -> Confirmed -> Processing -> Shipped -> Delivered, with Cancelled reachable from any state before Shipped. Nothing else in the app tracks order state independently.

07

Indexes

Firestore builds simple single-field indexes automatically. A **composite index** is required the first time a query filters on one field and sorts or filters on another — for example, "products in this category, sorted by price" or "variants where stock <= threshold, ordered by category." The first time such a query runs, Firestore's own error message contains a direct link that creates the exact index needed; click it and wait roughly a minute. Expect to do this a handful of times while building the Shop filters and the Inventory table's sort options — it is normal, not a bug.

08

Backups & data lifecycle

- Turn on scheduled Firestore exports to Cloud Storage (Firebase console -> Firestore -> Backups) — daily is enough for a store this size.
- Actually restore a backup into a test project at least once before launch. A backup nobody has restored is a guess, not a plan.
- stockLogs and assistantLogs grow forever by design — they are the audit trail and the AI transcript. Revisit retention (e.g. archive logs older than 12 months) once volume makes it worth the effort, not before.
- Order documents are never deleted, only ever moved to Cancelled — a full order history is both a legal record and the underlying data for GST invoicing.

Where the access rules live

This document describes what the data looks like. The full Firestore and Cloud Storage Security Rules that control who can read and write each collection are in the companion Security Rules & Compliance Document — the two are meant to be read together.

Database Document

The Cloud Firestore data model for Raj Raani Collections — every collection, its fields, sample documents, required indexes, and the data-integrity habits the app is built around.

Brand	Raj Raani Collections
Database	Cloud Firestore, region asia-south1 (Mumbai)
Security rules	See the companion Security Rules & Compliance Document

01

Document database, not relational

Firestore stores collections of documents, not tables of rows. There are no joins — a screen that needs data from two places either nests it as a subcollection, denormalizes a copy of the field it needs, or makes two reads. The model below makes that trade-off deliberately in a couple of places, each one called out.

02

Collection map

```
users/{uid}                fullName, phone, email, role, createdAt
|-- addresses/{addressId}  fullName, phone, line1, line2, city, state, pincode,
  isDefault
|-- cart/{itemId}          productId, variantId, quantity
|-- wishlist/{productId}   addedAt

categories/{categoryId}     name, slug, description, imageUrl, sortOrder, isActive

products/{productId}        categoryId, name, slug, description, price,
  compareAtPrice,
                                fabric, workType, setIncludes, weightGrams,
                                isCustomizable, leadTimeDays,
                                isFeatured, isBestSeller, status, images: [{url,
                                alt}]

productVariants/{variantId} productId, productName, categoryId, size, color, sku,
  stock, lowStockThreshold, priceOverride, updatedAt

orders/{orderId}            orderNumber, userId, status, subtotal, deliveryCharge,
  total,
                                paymentMethod, paymentStatus,
                                razorpayOrderId, razorpayPaymentId,
                                shiprocketOrderId, awbNumber, courierName,
                                shippingAddress (map), items: [{...}], placedAt

stockLogs/{logId}           variantId, productId, previousStock, newStock,
  reason, changedBy, changedAt

assistantLogs/{logId}       sessionId, userId, messages: [...], createdAt
```

03

Why productVariants is its own flat collection

Sizes and colors could have been stored as an array field inside each product document — but Firestore cannot run a range query (like "stock is at or below 3") inside an array of maps across many documents. Pulling every variant into its own top-level collection, with the parent product's name and category copied onto each row, makes the owner's inventory table a single, fast, sortable, filterable query — where `stock <= lowStockThreshold` — instead of an in-app scan of every product. The cost is a small one: if a product's name changes, the denormalized `productName` on its variants needs updating alongside it, which the admin

product editor does automatically in the same write.

04

Field reference

products/{productId}

FIELD	TYPE	NOTES
categoryId	reference	Which category this belongs to
name, slug	string	Slug drives the /product/[slug] URL — unique, lowercase, hyphenated
price, compareAtPrice	number	compareAtPrice shows a strikethrough when higher than price
fabric, workType, setIncludes	string	Silk/georgette/net · zari/zardozi/sequin · what's included in the set
weightGrams	number	What Shiprocket charges freight on
isCustomizable, leadTimeDays	boolean, number	A made-to-order piece skips the delivery-estimate badge and shows a lead time instead
isFeatured, isBestSeller	boolean	Powers Home sections and the /shop?filter= views
status	string	'draft' or 'published' — drafts never appear on the storefront
images	array of maps	{url, alt} — url points at Cloud Storage

productVariants/{variantId}

FIELD	TYPE	NOTES
productId, productName, categoryId	reference / string	Denormalized so the inventory table needs no join
size, color, sku	string	sku is the human-facing code used in Excel import/export
stock	number	The number the whole inventory system reads and writes
lowStockThreshold	number	Default 3 — anything at or below shows an amber badge
priceOverride	number, nullable	Rare — a specific size/color priced differently from the product default

orders/{orderId}

FIELD	TYPE	NOTES
orderNumber	string	Human-facing — what the customer quotes on WhatsApp or /track-order

FIELD	TYPE	NOTES
userId	reference, nullable	null for a guest order
status	string	New / Confirmed / Processing / Shipped / Delivered / Cancelled
paymentMethod, paymentStatus	string	'razorpay' or 'cod'; paymentStatus tracks pending/paid/refunded
razorpayOrderId, razorpayPaymentId	string	Set once payment is created / captured
shiprocketOrderId, awbNumber, courierName	string	Set once the order is pushed to Shiprocket
shippingAddress	map	A full copy of the address at the time of order — never a reference to a mutable address doc
items	array of maps	name and price are snapshotted at purchase time — see section 06

05

Sample documents

Realistic documents, not placeholders — this is what a working catalog and a placed order actually look like.

products/lh_royal_rose

```
{
  "categoryId": "cat_bridal",
  "name": "Royal Rose Bridal Lehenga",
  "slug": "royal-rose-bridal-lehenga",
  "price": 42999,
  "compareAtPrice": 48999,
  "fabric": "Silk with zari embroidery",
  "workType": "Zari, zardozi",
  "setIncludes": "Lehenga, unstitched blouse, dupatta",
  "weightGrams": 2600,
  "isCustomizable": true,
  "leadTimeDays": 21,
  "isFeatured": true,
  "isBestSeller": false,
  "status": "published",
  "images": [
    { "url": "https://.../royal-rose-1.jpg", "alt": "Royal Rose, front" }
  ]
}
```

productVariants/var_lh_royal_rose_m_maroon

```
{
  "productId": "lh_royal_rose",
  "productName": "Royal Rose Bridal Lehenga",
  "categoryId": "cat_bridal",
  "size": "M",
  "color": "Maroon",
  "sku": "RRC-BR-001-M-MRN",
  "stock": 3,
  "lowStockThreshold": 3,
  "priceOverride": null,
  "updatedAt": "2026-09-10T11:20:00Z"
}
```

orders/ord_20260910_0042 — orderNumber "RRC-1042"

```
{
  "orderNumber": "RRC-1042",
  "userId": null,
  "status": "Confirmed",
  "subtotal": 42999,
  "deliveryCharge": 0,
  "total": 42999,
  "paymentMethod": "razorpay",
  "paymentStatus": "paid",
  "razorpayOrderId": "order_XXXXXXXX",
  "razorpayPaymentId": "pay_XXXXXXXX",
  "shippingAddress": {
    "fullName": "Priya Sharma", "phone": "+91XXXXXXXXXX",
    "line1": "12 Sector 18", "city": "Noida", "state": "Uttar Pradesh", "pincode":
      "201301"
  },
  "items": [
    { "productId": "lh_royal_rose", "variantId": "var_lh_royal_rose_m_maroon",
      "nameSnapshot": "Royal Rose Bridal Lehenga", "priceSnapshot": 42999, "quantity":
        1 }
  ],
  "placedAt": "2026-09-10T11:22:04Z"
}
```

06

Two habits that keep the data honest

1. **Price and name are snapshotted** onto order.items at the moment of purchase. If a product's price changes next month, every past invoice still reads correctly — it is not silently rewritten.
2. **order.status is the single source of truth** for the whole fulfillment workflow: New -> Confirmed -> Processing -> Shipped -> Delivered, with Cancelled reachable from any state before Shipped. Nothing else in the app tracks order state independently.

07

Indexes

Firestore builds simple single-field indexes automatically. A **composite index** is required the first time a query filters on one field and sorts or filters on another — for example, "products in this category, sorted by price" or "variants where stock <= threshold, ordered by category." The first time such a query runs, Firestore's own error message contains a direct link that creates the exact index needed; click it and wait roughly a minute. Expect to do this a handful of times while building the Shop filters and the Inventory table's sort options — it is normal, not a bug.

08

Backups & data lifecycle

- Turn on scheduled Firestore exports to Cloud Storage (Firebase console -> Firestore -> Backups) — daily is enough for a store this size.
- Actually restore a backup into a test project at least once before launch. A backup nobody has restored is a guess, not a plan.
- stockLogs and assistantLogs grow forever by design — they are the audit trail and the AI transcript. Revisit retention (e.g. archive logs older than 12 months) once volume makes it worth the effort, not before.
- Order documents are never deleted, only ever moved to Cancelled — a full order history is both a legal record and the underlying data for GST invoicing.

Where the access rules live

This document describes what the data looks like. The full Firestore and Cloud Storage Security Rules that control who can read and write each collection are in the companion Security Rules & Compliance Document — the two are meant to be read together.